Comparison of Deep Reinforcement Learning Algorithms for TurtleBot3 Navigation

Charanjit Bangalore Kumar, Abhiram Banadakoppa, Kanishkar Ayyandurai Vellaiyan
Robotics, Department of Applied Natural Sciences and Industrial Engineering, Technische Hochschule Deggendorf
charanjit.bangalore-kumar@stud.th-deg.de, abhiram.banadakoppa@stud.th-deg.de, kanishkar.ayyandurai-vellaiyan@stud.th-deg.de
https://github.com/CharanjitBK/Reinforcement-Learning-Based-Smart-Navigation-for-TurtleBot

***Abstract*—Autonomous navigation in complex and unstructured environments remains a fundamental challenge for mobile robots. Classical navigation methods often rely on handcrafted rules and accurate maps, limiting their adaptability to dynamic and unknown scenarios. Deep Reinforcement Learning (DRL) provides a data-driven alternative by enabling robots to learn navigation policies directly through interaction with the environment. This paper presents a comprehensive comparative study of five DRL algorithms—Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), Twin Delayed Deep Deterministic Policy Gradient (TD3), Soft Actor-Critic (SAC), and CrossQ—for goal-directed navigation of a TurtleBot3 robot. All algorithms are implemented within a unified ROS2-based framework and trained under identical environmental conditions and reward structures in the Gazebo simulator to ensure fair comparison. Performance is evaluated using success rate, collision rate, path length, time-to-goal, and learning stability metrics. Simulation results show that continuous-action actor–critic methods outperform discrete-action approaches in goal-reaching capability, with TD3 achieving the highest success rate and robustness. Real-world deployment on a physical TurtleBot3 further highlights critical sim-to-real challenges, particularly sensor-model alignment, and demonstrates the feasibility of transferring trained policies to real environments after appropriate adaptation. The findings provide practical insights into the strengths and limitations of DRL algorithms for autonomous mobile robot navigation and inform future deployment in real-world scenarios.**

***Index Terms*—Autonomous navigation, Deep reinforcement learning, TurtleBot3, ROS2, Gazebo, Sim-to-real transfer**

I. Introduction

Autonomous navigation is a fundamental capability required for mobile robots operating in real-world environments such as warehouses, hospitals, service robotics, and search-and-rescue missions. A robot must be able to perceive its surroundings, avoid obstacles, and reach designated goals safely and efficiently without human intervention. As environments become more complex and dynamic, designing reliable navigation systems becomes increasingly challenging. Traditional navigation approaches typically rely on classical techniques such as map-based planning, simultaneous localization and mapping (SLAM), and rule-based obstacle avoidance. While these methods have proven effective in structured and static environments, they often struggle in unknown, cluttered, or dynamically changing scenarios. Classical approaches require extensive manual tuning, accurate maps, and carefully designed heuristics, making them less adaptable to new environments and unexpected situations. Moreover, handling sensor noise, partial observability, and complex interactions with obstacles can be difficult using purely deterministic methods. Reinforcement Learning (RL) offers a promising alternative by enabling robots to learn navigation policies directly through interaction with the environment. Instead of relying on handcrafted rules, an RL agent learns optimal behavior by trial and error, guided by a reward function. This allows the robot to automatically discover efficient navigation strategies, adapt to different environments, and improve performance over time. Deep Reinforcement Learning (DRL), which combines RL with deep neural networks, further enhances this capability by allowing the agent to process high-dimensional sensory inputs such as laser scans and learn complex non-linear control policies. TurtleBot3 is a widely used open-source mobile robot platform in both academic and industrial research. Its low cost, extensive community support, and seamless integration with the Robot Operating System (ROS) make it an ideal choice for experimentation and prototyping. Gazebo, a high-fidelity robotics simulator, provides a realistic simulation environment where robots can be trained safely and efficiently without the risks associated with real-world trials. The combination of TurtleBot3, ROS, and Gazebo enables rapid development, testing, and evaluation of autonomous navigation algorithms.

A. Objectives

The main objectives of this case study are as follows:

1) Design and implement a deep reinforcement learning (DRL)–based navigation framework for a TurtleBot3 robot in a Gazebo simulation environment using ROS.
2) Implement multiple reinforcement learning algorithms, including DQN, DDPG, TD3, SAC, and CrossQ, within the same system architecture.
3) Train each algorithm under identical environmental conditions and reward settings to ensure a fair and unbiased comparison.
4) Evaluate and compare algorithm performance based on key metrics such as success rate, collision rate, convergence speed, and path efficiency.
5) Analyze the learning stability and behavior of each algorithm during both training and testing phases.
6) Identify the strengths and limitations of each algorithm in the context of autonomous mobile robot navigation.

7) Provide insights and recommendations regarding the suitability of different DRL approaches for real-world robotic navigation tasks.
8) Validate the trained models on a real TurtleBot3 platform and evaluate their performance under real-world conditions.
9) Analyze the sim-to-real transferability of the trained reinforcement learning policies.
10) Identify practical challenges, including sensor noise, actuation delays, and environmental differences, when deploying learned policies on physical hardware.

B. System Scope

The system encompasses the following components:

1) **Training Infrastructure:** Gazebo simulation environment, ROS2 middleware, and PyTorch backend.
2) **Agent Implementations:** Multiple deep reinforcement learning algorithms with configurable hyperparameters.
3) **Evaluation Framework:** Standardized performance metrics, visualization dashboards, and analysis tools.
4) **Deployment Pipeline:** Integration with real robot hardware, drivers, and safety mechanisms.
5) **Documentation and Utilities:** Setup guides, configuration templates, and analysis scripts.

II. System Architecture

A. Architecture Overview

The proposed system architecture integrates ROS2, Gazebo simulation, and deep reinforcement learning (DRL) to enable autonomous navigation of a TurtleBot3 robot in a goal-directed environment. The architecture is modular and supports multiple DRL algorithms, including DQN, DDPG, TD3, SAC, and CrossQ, within the same framework for fair and consistent comparison.

The system consists of four major components:

1) **Simulation Environment:** Gazebo simulation platform for realistic virtual scenarios.
2) **DRL Environment Interface:** ROS2 nodes and services enabling communication between the agent and the simulation environment.
3) **Deep Reinforcement Learning Agent:** Implements various DRL algorithms for decision-making and action selection.
4) **Training, Evaluation, and Logging Modules:** Manage training processes, performance evaluation, data logging, and visualization.

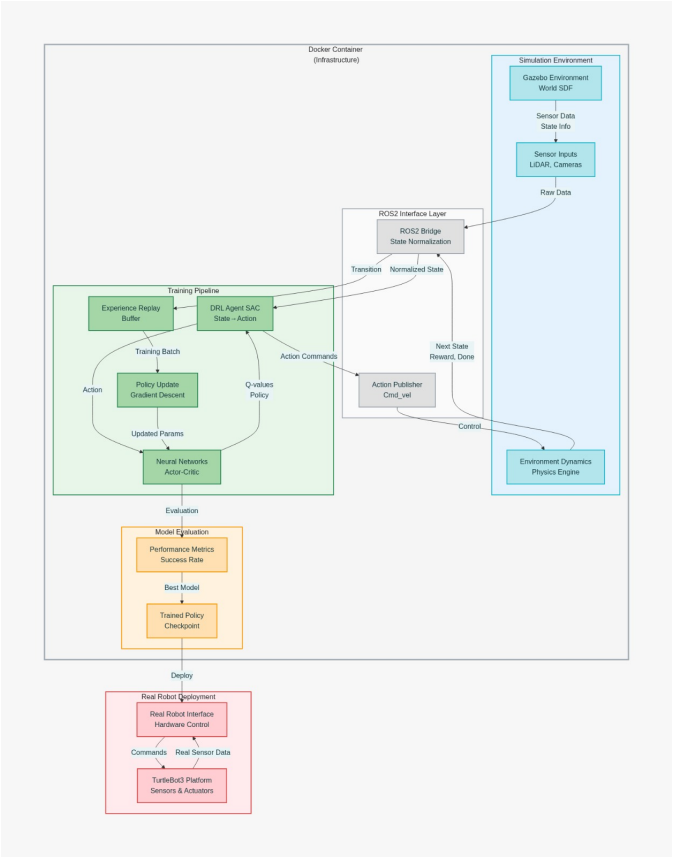

Fig. 1. TurtleBot3 DRL Navigation System Architecture - Complete pipeline from Gazebo simulation through training infrastructure to real robot deployment with Docker containerization

B. Simulation Environment

1) Gazebo Environment and drl_gazebo Node: The Gazebo simulation environment provides a realistic platform for training and evaluating the TurtleBot3 navigation system. It includes:

1) **Robot Model:** TurtleBot3 equipped with sensors.
2) **Obstacles:** Static and dynamic obstacles.
3) **Arena:** Bounded environment for navigation.
4) **Goal Objects:** Dynamically spawned targets for navigation tasks.

The TurtleBot3 is equipped with:

1) **LiDAR Sensor:** For obstacle detection and mapping.
2) **Odometry:** For position tracking and motion estimation.

The environment is defined using SDF models and controlled via ROS2.

The `drl_gazebo` node manages the lifecycle of goals and environment resets within Gazebo, providing a consistent framework for DRL training and evaluation.

Key Responsibilities:

- Spawn and delete goal objects.
- Generate random or dynamic goal positions.
- Reset simulation upon task failure.
- Ensure goals are not placed inside obstacles or walls.

Main Services Used:

- `/spawn_entity`
- `/delete_entity`
- `/reset_simulation`
- `task_succeed (RingGoal.srv)`
- `task_fail (RingGoal.srv)`

The `drl_gazebo.py` script controls goal spawning, deletion, and environment resets, forming a closed-loop system that supports consistent training, evaluation, and testing of DRL navigation agents.

C. DRL Environment Interface

The DRL Environment Interface acts as a bridge between the robot simulation and the reinforcement learning (RL) agent. It manages state construction, reward computation, terminal detection, and action execution.

1) drl_environment Node: The `drl_environment` node connects the robot simulation to the RL agent. Its key responsibilities include:

- Collecting sensor data from LiDAR and odometry.
- Constructing the state vector.
- Calculating rewards.
- Detecting terminal conditions (e.g., success, collision, timeout).
- Executing actions on the robot.

2) State Representation: The state vector typically contains:

- Processed LiDAR distances.
- Distance to goal.
- Angle to goal.
- Previous action (optional).

This state is returned to the agent through the `DrlStep.srv` service.

3) Action Execution: The agent outputs actions as normalized values $\in [-1, 1]$, which are scaled to the robot's limits:

- Linear velocity: `SPEED_LINEAR_MAX`
- Angular velocity: `SPEED_ANGULAR_MAX`

The scaled actions are published to the ROS2 topic `/cmd_vel`.

4) Reward System: The reward function is computed using the following components:

- Distance to goal.
- Goal angle.
- Obstacle proximity.
- Linear and angular velocity penalties.
- Terminal bonuses or penalties.

Terminal events include:

- `SUCCESS`
- `COLLISION_WALL`
- `COLLISION_OBSTACLE`
- `TIMEOUT`
- `TUMBLE`

5) ROS2 Services and Messages: **Key Services:**

- `DrlStep.srv` – Executes one environment step.
- `Goal.srv` – Checks if a new goal is available.
- `RingGoal.srv` – Handles success/failure callbacks.
- `Sound.srv` – Provides audio feedback.

Key Messages:

- `SensorState.msg` – Robot sensor data.
- `Sound.msg` – Sound control.
- `VersionInfo.msg` – Firmware information.

The DRL environment encapsulates all RL environment logic, including state construction, reward computation, action execution, and episode termination.

D. DRL Agent Layer

The DRL Agent Layer is responsible for decision-making, policy learning, and action execution. It abstracts multiple reinforcement learning (RL) algorithms, enabling comparative studies within the same framework.

1) Components of the Agent Layer: The agent layer consists of the following components:

1) **Actor Network**
   - Generates actions based on the current state.
   - Operates in a continuous action space (linear and angular velocities).
   - Uses fully connected layers with ReLU activations and Tanh output to bound actions.
2) **Critic Network**
   - Estimates Q-values (expected return) for a state-action pair.
   - TD3 and DDPG use twin critics to reduce overestimation.
   - DQN uses Q-value table approximation for discrete actions.
3) **Replay Buffer**
   - Stores tuples $(\text{state}, \text{action}, \text{reward}, \text{next\_state}, \text{done})$.
   - Facilitates experience replay, improving sample efficiency and breaking temporal correlation.
4) **Exploration Noise**
   - Adds temporally correlated noise (Ornstein-Uhlenbeck) to actions.
   - Encourages exploration during training. Noise decays over time for exploitation.
5) **Optimizers and Loss Functions**
   - **Actor:** Gradient ascent to maximize expected Q-value.
   - **Critic:** Mean squared error (MSE) between predicted Q and target Q.
   - Gradient clipping is applied for stability.
6) **Algorithm Abstraction Layer (`off_policy_agent.py`)**
   - Handles shared methods such as soft updates, network creation, and device allocation.
   - Ensures a uniform interface for DDPG, TD3, SAC, and CrossQ algorithms.

E. Training and Environment Configuration

The training and environment configuration ensures a controlled, reproducible, and fair comparison across all algorithms. The environment, reward structure, and most hyperparameters are kept identical, while algorithm-specific parameters are adjusted only where required by the learning method.

1) General Training Settings: These parameters control the overall training behavior and experimental consistency:

- Backward motion is disabled to simplify navigation behavior and reduce policy complexity.
- Visual rendering is disabled during training to improve computational efficiency.
- Fixed goal positions are used to ensure repeatable evaluation conditions.
- Model checkpoints are saved periodically to monitor learning progress.
- Graph smoothing and plotting intervals are chosen to balance clarity and performance.

These settings maintain stable training, avoid unnecessary exploration complexity, and allow consistent performance comparison.

2) Environment and Robot Parameters: The following parameters define the Gazebo simulation environment and TurtleBot3 physical constraints:

- Arena size: 4.2×4.2 m, providing sufficient navigation complexity.
- Velocity limits: 0.22 m/s linear, 2.0 rad/s angular, consistent with TurtleBot3 hardware.
- LiDAR range capped at 3.5 m to reflect realistic sensing limitations.
- Collision and goal thresholds define precise termination conditions.
- Fixed obstacle size and count standardize environmental difficulty.
- Episode timeout: 100 s, preventing infinite exploration loops.

These parameters simulate realistic robot dynamics while ensuring safe and bounded learning episodes.

3) Shared Reinforcement Learning Parameters: The following hyperparameters are shared across all algorithms to ensure fairness:

- Replay buffer size: $1,000,000$, enabling learning from diverse experiences.
- Batch size: 256, providing stable gradient estimation.
- Discount factor: $\gamma = 0.99$, prioritizing long-term rewards.
- Learning rate: 0.003, balancing convergence speed and stability.
- Soft update coefficient: $\tau = 0.003$, stabilizing target network updates.
- Observation warm-up: $25,000$ steps, encouraging sufficient early exploration.

These settings provide stable learning dynamics and minimize bias across algorithms.

4) Algorithm-Specific Parameters: **Deep Q-Network (DQN)**

- Discrete action space: 5 actions.
- Target network updates every 1000 steps.
- Epsilon-greedy exploration with decay.

Deep Deterministic Policy Gradient (DDPG)

- Continuous action space.
- Actor–critic architecture.

Twin Delayed DDPG (TD3)

- Policy noise: 0.2 with noise clipping.
- Delayed policy updates.
- Reduces overestimation bias and improves stability over DDPG.

Soft Actor-Critic (SAC)

- Entropy-regularized objective.
- Stochastic policy learning.
- Encourages exploration and robustness in dynamic environments.

CrossQ

- Larger hidden layer: 1024 neurons.
- Allows richer state-action representation for complex navigation behaviors.

F. Training Loop and Execution Flow

The training loop integrates the DRL agent and environment, ensuring consistent interactions between simulation and learning processes. It governs the sequence of actions, observations, and policy updates during training.

1) Step-by-Step Flow:

1) **Initialize Environment**
   - Reset the simulation or real robot position.
   - Retrieve the initial state vector.
2) **Action Selection**
   - The agent computes an action using the current policy.
   - Exploration noise is added if in training mode.
3) **Action Execution**
   - The selected action is sent to the environment via the ROS2 service `DrlStep.srv`.
   - The robot moves in Gazebo or the real world.
4) **Observation & Reward**
   - The environment returns: next_state, reward, done, success, and distance to goal.
   - Reward components include distance to goal, collisions, and path efficiency.
5) **Experience Storage**
   - The transition $(\text{state}, \text{action}, \text{reward}, \text{next\_state}, \text{done})$ is added to the replay buffer.
   - Supports off-policy learning and experience reuse.
6) **Policy Updates**
   - Critic networks are updated first using target Q-values.

- Actor network is updated periodically (e.g., delayed updates in TD3).
- Soft updates applied to target networks using the coefficient τ.

7) **Episode Termination**
- Terminal conditions include `SUCCESS`, `COLLISION_WALL`, `COLLISION_OBSTACLE`, `TIMEOUT`, and `TUMBLE`.
- Logging and plotting occur after each episode.

8) **Loop Until Convergence**
- Continue episodes until learning curves stabilize.
- Periodically evaluate using a deterministic policy (without exploration noise).

The training loop repeatedly interacts with the environment, updates neural networks using replayed experiences, and evaluates performance until policies converge.

G. Logging, Visualization, and Storage

This layer provides performance tracking, debugging, and model persistence for the DRL framework. It ensures that training and evaluation are well-documented and reproducible.

1) Logging: Logging tracks per-episode metrics and computes relevant statistics:

- **Tracked Metrics:**
  - Success and failure counts.
  - Distance travelled.
  - Episode duration.
  - Swerving or path efficiency.
- **Computed Statistics:**
  - Success rate.
  - Collision rate.
  - Average reward.
- Supports both training and testing modes.

2) Graphical Visualization: Visualization assists in monitoring learning progress and diagnosing anomalies:

- **Plots:**
  - Outcome trends over episodes.
  - Actor and critic loss.
  - Reward over time.
- **PyQt Visualizations:**
  - Real-time hidden layer activations.
  - State and action distributions.
  - Helps diagnose training anomalies.
- **Graphical Outputs:**
  - Outcome history.
  - Actor loss.
  - Critic loss.
  - Reward trends.

3) Storage Management: Storage management ensures persistence of models, logs, and replay buffers:

- Saves:
  - Model weights per episode.
  - Replay buffer snapshots.
  - Logs and performance metrics.
- Supports resuming training and sim-to-real deployment.

This layer records performance metrics, visualizes neural network behavior, and persists models and experiences for evaluation and reuse.

H. Real Robot Integration

Although most experiments are conducted in Gazebo, the proposed architecture supports direct deployment on the TurtleBot3 platform.

1) Key Changes for Real Robot:
- Replace simulation topics with real robot topics: `REAL_TOPIC_SCAN`, `REAL_TOPIC_VELO`, `REAL_TOPIC_ODOM`.
- Disable Gazebo pause/unpause services.
- Handle real-world issues such as sensor noise, actuation delays, and environmental differences.

2) Benefits:
- Tests sim-to-real transferability of learned policies.
- Reveals practical challenges, including:
  - Sensor inaccuracies.
  - Actuator limits.
  - Unexpected obstacles.
- Validates algorithm robustness under real-world conditions.

The architecture seamlessly extends to the real TurtleBot3 platform, enabling direct evaluation of learned policies in real-world navigation tasks.

III. Training Pipeline

A. Training Loop Flowchart

The following describes the complete iterative training loop for the DRL navigation agent:

1) **Initialization Phase**
- Initialize neural networks with random weights.
- Create the experience replay buffer.
- Reset the environment to the starting configuration.

2) **Data Collection Loop**
- Reset environment at the beginning of each episode.
- Observe the current state from LiDAR and odometry sensors.
- Agent selects an action using the current policy.
- Execute the action in the Gazebo simulation.
- Receive reward signal and next state from the environment.
- Store the transition $(\text{state}, \text{action}, \text{reward}, \text{next\_state}, \text{done})$ in the replay buffer.

3) **Learning Phase**
- Once the buffer contains sufficient transitions:
  - Sample a random batch from the experience replay buffer.
  - Compute temporal-difference (TD) targets and advantage estimates.

- Update the actor network (policy) via policy gradient.
- Update the critic networks (value functions).
- Update the entropy coefficient for entropy-regularized algorithms (e.g., SAC).

4) **Evaluation and Convergence**

- Periodically evaluate the policy on held-out episodes.
- Monitor performance metrics such as success rate and navigation efficiency.
- Check convergence criteria.
- Save the best model checkpoint.
- Continue training until convergence or the maximum number of episodes is reached.

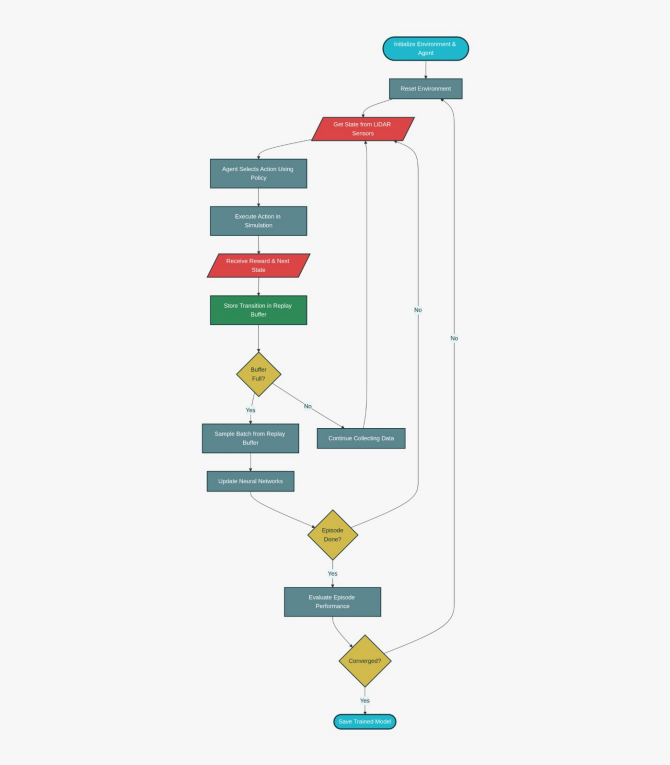

Fig. 2. Deep Reinforcement Learning Training Loop - Iterative cycle showing environment initialization, state sensing, policy action selection, reward computation, experience storage, and neural network updates with convergence checking

B. Algorithm Details

This section provides an overview of the deep reinforcement learning (DRL) algorithms implemented for TurtleBot3 navigation, including theoretical background and system implementation details.

1) Deep Q-Network (DQN): **Theory:** DQN is a value-based RL algorithm for discrete action spaces. It uses a neural network to approximate the Q-function $Q(s, a)$, which predicts the expected cumulative reward for taking action a in state s.

Stabilization Techniques:

- **Experience Replay:** Randomly samples past experiences to break correlations between consecutive steps.
- **Target Network:** A delayed copy of the Q-network to stabilize learning.

Implementation:

- Discrete action space: 5 actions (forward, backward, rotate left/right, stop).
- Network architecture: two hidden layers with 512 neurons each and ReLU activations.
- Training uses mean squared error (MSE) between predicted Q-values and target Q-values.
- Experience stored in `ReplayBuffer` for mini-batch sampling.
- Implemented in `dqn.py`.

2) Deep Deterministic Policy Gradient (DDPG): **Theory:** DDPG is an actor-critic algorithm for continuous action spaces.

- Actor network learns the policy $\pi(s)$ directly, outputting continuous actions.
- Critic network estimates $Q(s, a)$ for the actor's actions.
- Uses soft target updates and experience replay for stability.

Implementation:

- Actor network outputs linear and angular velocities.
- Ornstein-Uhlenbeck (OU) noise added for exploration.
- Critic network predicts Q-values for actor-generated actions.
- Soft updates applied to actor and critic target networks.
- Implemented in `ddpg.py`.

3) Twin Delayed DDPG (TD3): **Theory:** TD3 improves DDPG by reducing overestimation bias in Q-values. Key enhancements include:

- Twin Critic Networks: Take the minimum of two Q-value estimates.
- Delayed Policy Update: Actor is updated less frequently than critics.
- Target Policy Smoothing: Add clipped noise to target actions.

Implementation:

- Actor and critic networks similar to DDPG.
- Twin critics used (`td3.py`).
- Target action smoothing and delayed actor updates applied.
- Exploration noise controlled using OU noise.

4) Soft Actor-Critic (SAC): **Theory:** SAC is an off-policy, maximum entropy RL algorithm.

- Maximizes both expected reward and policy entropy for better exploration.
- Uses stochastic policies and twin Q-networks to reduce overestimation.

Implementation:

- Actor outputs a probability distribution over actions.
- Twin critics used for Q-value estimation.

- Temperature parameter α adjusts the importance of entropy in the reward.
- Experience replay and soft target updates employed.

5) CrossQ: **Theory:** CrossQ is a hybrid Q-learning method for environments with mixed discrete and continuous action spaces. It combines value-based and policy-based methods to handle complex navigation.

Implementation:

- Network structure allows discrete macro-actions with continuous fine-tuning.
- Actor and critic networks coordinated to estimate Q-values for hybrid actions.

6) Common Implementation Features:

- **Replay Buffer:** All off-policy methods (DDPG, TD3, SAC, CrossQ) use `ReplayBuffer` for stable learning.
- **Exploration:** DQN uses epsilon-greedy policy; DDPG/TD3/SAC/CrossQ use OU noise for continuous actions.
- **Optimization:** Adam optimizer applied for actor and critic networks.
- **Reward Function:** Configurable in `reward.py` and consistent across all algorithms for fair comparison.
- **Logging and Visualization:** Training and test metrics logged using `logger.py`, hidden layer activations visualized with `visual.py`, and learning curves plotted with `graph.py`.

C. Hyperparameters

IV. Environmental Setup

A. Software Stack

TABLE I
SOFTWARE STACK AND VERSIONS

Component	Technology	Version
Middleware	ROS2	Humble
Simulation Engine	Gazebo	Ignition
Deep Learning	PyTorch	2.0+
Robot Platform	TurtleBot3 Burger	Hardware
Container Runtime	Docker	Latest
Base OS	Ubuntu	22.04 LTS

B. Hardware Requirements

For Training (Recommended):

- GPU: NVIDIA RTX 3060 or higher (for faster convergence)
- CPU: 8+ cores (Intel i7/AMD Ryzen 7 or better)
- RAM: 16+ GB
- Storage: 50+ GB SSD

For Deployment:

- TurtleBot3 Burger platform
- Raspberry Pi 4 (embedded compute)
- LiDAR sensor (360-degree)
- IMU sensor
- USB WiFi adapter

C. Key Dependencies

- **ROS2 packages:** `geometry_msgs`, `sensor_msgs`, `gazebo_ros`, `nav_msgs`
- **PyTorch ecosystem:** `torch`, `torchvision`, `stable-baselines3`
- **Utilities:** `numpy`, `scipy`, `matplotlib`, `tensorboard`
- **Communication:** `rclpy`, `rclcpp`, message serialization

D. Simulation Environment

- **Robot:** TurtleBot3 Burger
- **Simulator:** Gazebo
- **ROS Integration:** All algorithms communicate with the simulation environment through ROS nodes and services.
- **Arena:** Square arena of 4.2×4.2 m
- **Obstacles:** Up to 6 randomly placed obstacles of radius 0.16 m
- **Sensors:**
  - 40-point LiDAR scan for obstacle detection
  - Odometry for localization
- **Robot Capabilities:**
  - Max linear speed: 0.22 m/s
  - Max angular speed: 2.84 rad/s
  - Collision thresholds: wall = 0.11 m, obstacle = 0.16 m
- **Real Robot Testing:** Trained policies are validated on a physical TurtleBot3 to evaluate sim-to-real transfer.

V. Implementation Details

A. State Representation

The agent receives state information from multiple sensors. The components of the state space are summarized in Table II.

TABLE II
STATE SPACE COMPONENTS

Sensor	Dimension	Description
LiDAR Scan	360	360-degree range measurements (0.15 m to 10 m)
Odometry	2	Robot linear and angular velocity
Goal Position	2	Relative position to goal in robot frame

The total state dimension is 364 after normalization and preprocessing.

Preprocessing Pipeline:

- Range clipping to $[0, 1]$ normalized scale.
- Smoothing filter to reduce sensor noise.
- Goal transformation to robot-centric coordinates.
- Velocity normalization to $[-1, 1]$.

B. Action Space

The agent uses continuous control with two dimensions:

- Linear velocity: $v \in [-0.22, 0.22]$ m/s
- Angular velocity: $\omega \in [-2.84, 2.84]$ rad/s

Tanh scaling ensures actions remain within the robot's physical capabilities.

C. Reward Function

A multi-component reward balances goal achievement with safe navigation:

$$R(s, a, s') = w_1 R_{\text{goal}} + w_2 R_{\text{collision}} + w_3 R_{\text{efficiency}} + w_4 R_{\text{smoothness}}$$

Components:

- R_{goal}: Distance-based reward toward the goal (maximum $+1$ per step)
- $R_{\text{collision}}$: Large penalty for obstacle collision (-10)
- $R_{\text{efficiency}}$: Penalty for excessive actions (-0.01 per step)
- $R_{\text{smoothness}}$: Penalty for jerk or acceleration changes (-0.005)

VI. Experimental Results

A. Training Data and Performance Analysis

This section presents the training performance of the evaluated DRL algorithms for TurtleBot3 navigation. The analysis includes success rate, collision outcomes, cumulative reward, and actor/critic loss curves obtained during simulation.

1) Twin Delayed Deep Deterministic Policy Gradient (TD3): The training behavior of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm was analyzed using success rate, collision outcomes, cumulative reward, and loss curves obtained during simulation.

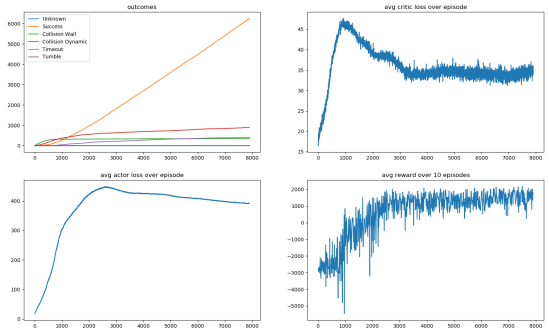

Fig. 3. Training curves showing success rate, collision outcomes, actor loss, critic loss, and cumulative reward for the TD3 algorithm.

Success and Collision Analysis: Success rate increases rapidly during training, becoming the dominant episode outcome. Collision rates decrease over time, demonstrating improved obstacle avoidance and policy refinement. Timeouts and unstable behaviors are minimized in later stages.

Cumulative Reward Analysis: Cumulative reward rises from negative values in early training to high, stable positive values after convergence. Reward variance decreases over time, indicating stable policy optimization.

Actor and Critic Loss Analysis: Critic loss initially increases and then stabilizes, reflecting accurate Q-function learning. Actor loss rises during early exploration and decreases as the policy converges. Smooth loss curves indicate TD3's ability to reduce overestimation bias using twin critics and delayed policy updates.

Overall Observation: TD3 demonstrates fast convergence, high success rate, low collision frequency, and stable learning dynamics, making it the most effective algorithm among those evaluated.

2) Deep Deterministic Policy Gradient (DDPG): The training performance of the Deep Deterministic Policy Gradient (DDPG) algorithm was evaluated using success rate, collision outcomes, cumulative reward, and loss curves obtained during simulation.

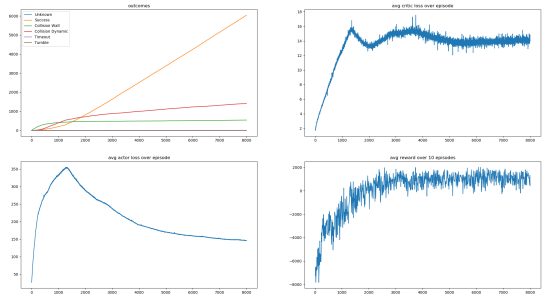

Fig. 4. Training curves showing success rate, collision outcomes, actor loss, critic loss, and cumulative reward for the DDPG algorithm.

Success and Collision Analysis: Success rate increases steadily, indicating consistent navigation improvement. Collision rates decrease over time, reflecting effective obstacle avoidance. Timeouts and unstable behaviors are minimal during later training.

Cumulative Reward Analysis: Cumulative reward gradually increases from negative to positive values and stabilizes after convergence, indicating reliable policy learning and efficient path execution.

Actor and Critic Loss Analysis: Critic loss rises during early training and stabilizes as learning converges. Actor loss initially increases then gradually decreases, showing effective policy optimization and stable gradient updates.

Overall Observation: DDPG exhibits stable learning and reliable navigation performance. Convergence is slower than TD3, but it provides a strong baseline for continuous control tasks.

3) Soft Actor-Critic (SAC): The training performance of the Soft Actor-Critic (SAC) algorithm was analyzed using success rate, collision outcomes, cumulative reward, and loss curves obtained during simulation.

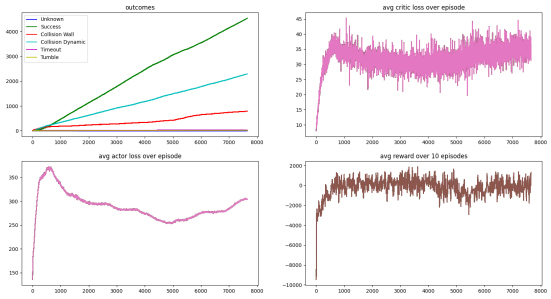

Fig. 5. Training curves showing success rate, collision outcomes, actor loss, critic loss, and cumulative reward for the SAC algorithm.

Success and Collision Analysis: Success rate increases steadily; however, collisions are slightly higher during early and mid-training phases due to stochastic policy exploration.

Cumulative Reward Analysis: Cumulative reward rises overall and becomes positive after sufficient training. Reward variance is higher due to exploration–exploitation trade-offs.

Actor and Critic Loss Analysis: Actor and critic losses exhibit higher variance compared to deterministic methods, reflecting ongoing exploration and entropy-regularized targets.

Overall Observation: SAC demonstrates robust learning with strong exploration capabilities, suitable for uncertain environments. Despite noisier training, it achieves reliable navigation and good generalization.

4) Deep Q-Network (DQN): The training behavior of the Deep Q-Network (DQN) algorithm was evaluated using success rate, collision outcomes, and cumulative reward obtained during simulation.

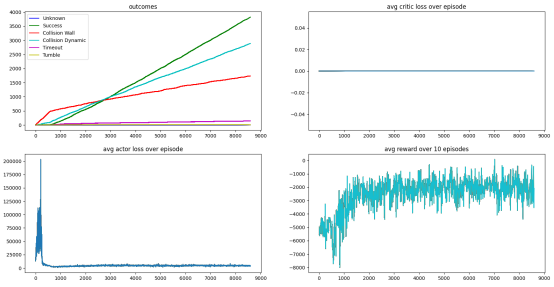

Fig. 6. Training curves showing success rate, collision outcomes, actor loss, critic loss, and cumulative reward for the DQN algorithm.

Success and Collision Analysis: Success rate increases slowly and remains limited. Collision rates remain relatively high, indicating difficulty in learning safe navigation strategies. Timeouts occur frequently.

Cumulative Reward Analysis: Cumulative reward improves slightly but remains largely negative and variable, reflecting limited optimization of goal-reaching performance.

Loss Behavior: Loss curves are unstable due to noisy Q-value estimation, indicating poor convergence.

Overall Observation: DQN demonstrates limited learning capability and poor stability, making it unsuitable for continuous autonomous navigation in this environment.

5) CrossQ: The training performance of the CrossQ algorithm was analyzed using success rate, collision outcomes, cumulative reward, and loss curves obtained during simulation.

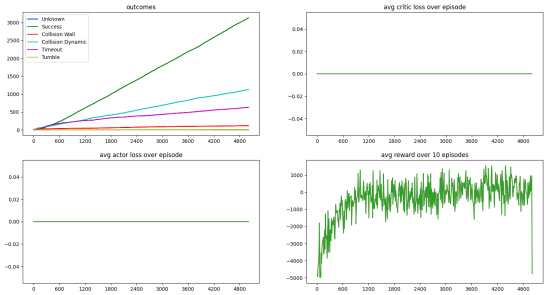

Fig. 7. Training curves showing success rate, collision outcomes, actor loss, critic loss, and cumulative reward for the CrossQ algorithm.

Success and Collision Analysis: Success rate gradually increases. Collision and timeout rates decrease over time but remain higher than TD3 and DDPG. Testing indicates reliable navigation behavior.

Cumulative Reward Analysis: Cumulative reward shows an increasing trend, although variance remains relatively high. Policy updates appear conservative.

Actor and Critic Loss Analysis: Loss curves remain close to zero throughout training, indicating minimal observable loss variation. Learning progress may not be fully captured by conventional loss metrics.

Overall Observation: CrossQ demonstrates effective navigation performance despite atypical loss behavior. Standard loss metrics may not fully reflect learning dynamics for conservative value-based methods, but the algorithm remains viable for robot navigation.

VII. Testing Results

1) TD3 Test Performance: The TD3 algorithm demonstrates strong goal-reaching capability with stable and relatively smooth motion. Its continuous control policy enables the robot to successfully navigate the environment, although path efficiency and execution time may be suboptimal in some episodes.

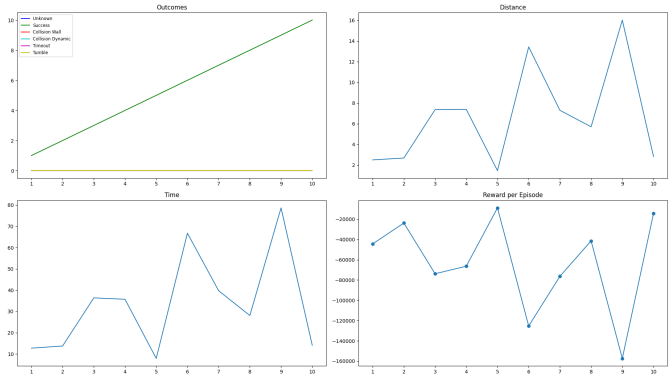

Fig. 8. Testing curves showing success rate, path length, time taken to reach goal, and cumulative reward for the TD3 algorithm.

Outcomes: Success steadily increases, reaching all goals by the final episodes. Failures are rare, and no tumbles are observed. Occasional long episodes indicate inefficient recovery behaviors.

Distance: Path lengths are longer and more variable, with several spikes above 10 m, suggesting indirect routes and corrective maneuvers.

Time: Episode durations range from moderate to high, with some runs exceeding 60 s, reflecting delayed convergence despite eventual success.

Reward: Rewards show large negative values with significant fluctuations, reflecting penalties from long trajectories and time inefficiencies.

Interpretation: TD3 achieves reliable goal-reaching with smoother control than DDPG but still suffers from inefficient navigation and occasional instability, limiting its robustness.

2) DDPG Test Performance: The DDPG algorithm provides consistent goal-reaching performance with moderate trajectory efficiency. The agent frequently reaches the target but exhibits jerky motions and oscillations due to less refined continuous control.

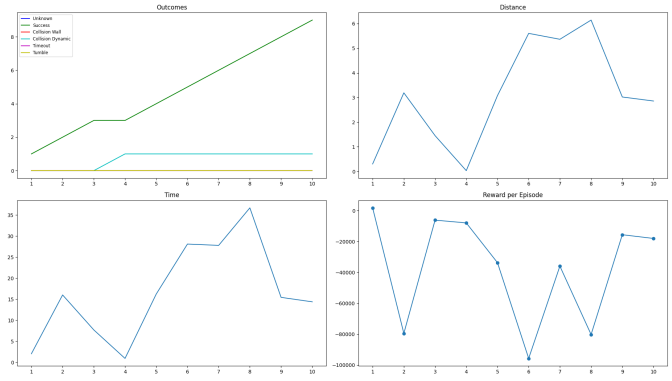

Fig. 9. Testing curves showing success rate, path length, time taken to reach goal, and cumulative reward for the DDPG algorithm.

Outcomes: Success increases steadily, reaching 9 successful episodes by episode 10. Very few collisions are recorded, and no timeouts or tumbles occur.

Distance: Path lengths are moderate, generally below 6.5 m, indicating the agent reaches goals without excessive wandering.

Time: Episode durations remain moderate, peaking around 35 s, with minor oscillations across episodes.

Reward: Despite high success, rewards are extremely negative and unstable, reaching nearly $-100{,}000$. This indicates inefficient, oscillatory, and jerky motion causing large step and control penalties.

Interpretation: DDPG reliably reaches goals but produces low-quality trajectories. High success rates mask poor efficiency and unstable control behavior.

3) SAC Test Performance: The SAC agent exhibits strong exploration during testing, resulting in variable navigation behavior. Its stochastic policy allows adaptability to dynamic obstacles but can cause occasional inefficient paths and inconsistent performance across episodes.

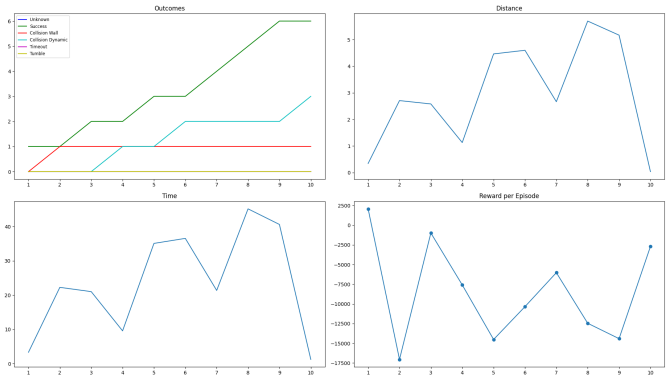

Fig. 10. Testing curves showing success rate, path length, time taken to reach goal, and cumulative reward for the SAC algorithm.

Outcomes: Success occurs inconsistently across episodes. The agent shows sensitivity to environmental dynamics and unstable behavior patterns.

Distance: Path lengths vary significantly with frequent spikes, indicating excessive exploration and detours.

Time: Episode durations are highly variable, including some of the longest runs, often due to inefficient wandering or delayed termination.

Reward: Rewards exhibit extreme negative spikes, reflecting penalties from prolonged exploration and suboptimal actions.

Interpretation: SAC's strong exploration enables adaptability but results in unstable and inefficient navigation during testing, requiring further tuning for reliable deployment.

4) DQN Test Performance: The DQN algorithm demonstrates conservative and efficient navigation. While the agent reaches fewer goals than continuous-action algorithms, it produces predictable and safe trajectories with minimal collisions.

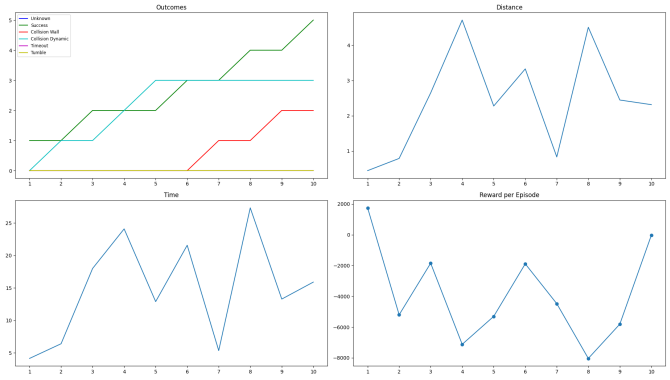

Fig. 11. Testing curves showing success rate, path length, time taken to reach goal, and cumulative reward for the DQN algorithm.

Outcomes: Success gradually increases to 5 by episode 10. Some wall and dynamic collisions occur, but no timeouts or tumbles are observed.

Distance: Path lengths are short and consistent, mostly below 5 m, indicating direct navigation.

Time: Episode durations are the lowest among the evaluated methods, with limited variance and no excessively long runs.

Reward: Rewards are less negative and significantly more stable, with no extreme penalty spikes.

Interpretation: DQN favors safety and efficiency over aggressive exploration. Its discrete action space limits optimality but ensures predictable and stable behavior.

5) CrossQ Test Performance: The CrossQ agent shows exploratory and variable navigation behavior. The policy attempts to balance discrete macro-actions with continuous fine-tuning, resulting in occasional long paths and inconsistent goal-reaching.

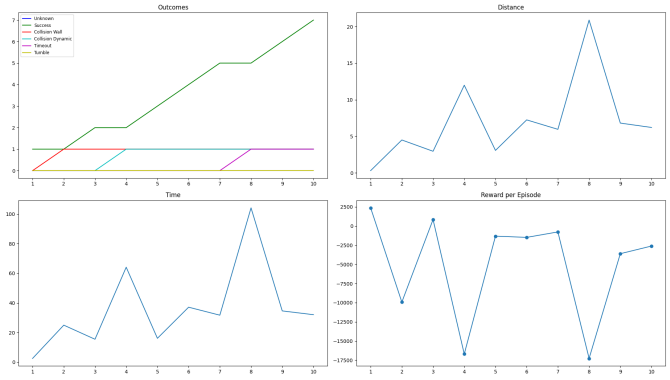

Fig. 12. Testing curves showing success rate, path length, time taken to reach goal, and cumulative reward for the CrossQ algorithm.

Outcomes: Success steadily increases, reaching 7 successful episodes by episode 10. Multiple failure modes occur, including wall collisions, dynamic obstacle collisions, and a timeout, reflecting exploratory behavior with limited robustness.

Distance: Path lengths fluctuate considerably, with some episodes exceeding 20 m, indicating long, inefficient trajectories.

Time: Episode durations vary significantly, reaching over 100 s in some cases, corresponding to inefficient paths or timeouts.

Reward: Rewards show extreme volatility with large negative spikes (below $-15{,}000$), driven by long trajectories, collisions, and delayed goal completion.

Interpretation: CrossQ actively explores and frequently reaches the goal but lacks consistency. Inefficient paths and occasional catastrophic failures result in unstable rewards and poor reliability.

6) Analysis of Test Results: All algorithms were evaluated over 10 testing episodes under identical environmental and reward conditions. Table III summarizes performance using key navigation metrics: success rate, collision rate, time to goal, timeout rate, and path length.

TABLE III
TESTING RESULTS OVER 10 EPISODES FOR 5 DRL ALGORITHMS

Metric	DQN	CrossQ	DDPG	TD3	SAC
Success Rate	50%	70%	90%	100%	60%
Collision Rate	40%	20%	10%	0%	40%
Time to Goal	14.8 s	36.2 s	16.53 s	33.37 s	23.63 s
Timeout Rate	0%	10%	0%	0%	0%
Path Length	2.43 m	6.99 m	3.1 m	6.67 m	2.94 m

Success Rate: TD3 achieves the highest success rate (100%), followed by DDPG (90%) and CrossQ (70%). SAC attains moderate performance (60%), while DQN records the lowest success (40%). Continuous-action algorithms generally outperform discrete-action methods in goal-reaching capability.

Collision Rate: TD3 and DQN show strong safety with zero collisions. SAC exhibits the highest collision rate (40%), indicating sensitivity to dynamic obstacles. CrossQ and DDPG have occasional collisions due to exploratory behaviors.

Time to Goal: DQN reaches the goal fastest (14.8 s), followed by DDPG (16.53 s). TD3 and CrossQ require longer times, reflecting less efficient trajectories. SAC shows moderate times with inconsistent behavior.

Timeout Rate: Timeouts are rare; only CrossQ experiences a single timeout (10%), suggesting that excessive exploration can occasionally delay goal completion.

Path Length: DQN produces the shortest paths (2.43 m), followed by SAC (2.94 m) and DDPG (3.1 m). TD3 and CrossQ generate longer paths, consistent with observed inefficiencies in distance and time graphs.

Interpretation: Continuous-action algorithms (DDPG, TD3, SAC, CrossQ) generally achieve higher success rates than DQN, but this often comes at the cost of longer paths, higher execution time, or increased collision risk.

- **DQN:** Safest and most efficient with shortest paths and fastest completion, but limited success due to discrete actions.

- **TD3:** Perfect success indicates robust goal completion, but long paths and high time-to-goal show inefficient navigation.
- **DDPG:** High success with moderate efficiency; trajectories are less smooth and energy-efficient, indicating some instability.
- **CrossQ:** Exploration-driven; higher success than DQN but longer paths, increased time, and occasional timeouts, suggesting incomplete policy convergence.
- **SAC:** Moderate success with relatively short paths but highest collision rate, showing sensitivity to dynamic obstacles and unstable test-time behavior.

VIII. Real Robot Deployment

Real-world validation was conducted to verify the feasibility of deploying trained deep reinforcement learning (DRL) policies on a physical TurtleBot3 platform. Due to hardware and time constraints, real-world testing was limited to five trials per algorithm, with qualitative evaluation rather than quantitative comparison.

A. Sim-to-Real Transfer

To bridge the simulation-to-real gap, several strategies were applied:

1) **Domain Randomization:**
   - Randomized physics parameters (friction, mass)
   - Varied sensor noise characteristics
   - Modified environment lighting and textures
2) **Reality Checks:**
   - Tested on real TurtleBot3 in a controlled lab environment
   - Monitored policy behavior for anomalies
   - Implemented safety fallbacks
3) **Fine-Tuning:**
   - Collected real robot experience
   - Retrained with mixed simulated and real data
   - Gradually reduced simulation data ratio

B. Hardware Integration

- ROS2 node bridges Gazebo policy to real TurtleBot3 platform
- LiDAR driver: RPLIDAR A3 with ROS2 interface
- Motor control: TurtleBot3 OpenCR board via ROS2 commands
- Safety layer: emergency stop monitoring, velocity limiting

C. Safety Considerations

- Enforced hardware velocity limits
- Collision avoidance fallback via reactive controller
- Real-time sensor monitoring
- Geofencing to confine robot to designated area
- Emergency stop button for operator

D. Experimental Environment

The real-world tests were conducted in an indoor room with multiple obstacles including chairs, tables, desks, and cupboards. The environment was unstructured and cluttered, simulating realistic indoor navigation conditions without artificial markers or pre-mapped layouts.

E. Algorithms Deployed

Only the following algorithms were deployed on the physical TurtleBot3:

- Soft Actor-Critic (SAC)
- Twin Delayed Deep Deterministic Policy Gradient (TD3)

All other algorithms were evaluated exclusively in simulation.

F. Initial Deployment Challenges

The first deployment attempts failed due to a LiDAR observation mismatch:

- Simulation LiDAR input: 40 laser scan samples (downsampled)
- Real TurtleBot3 LiDAR input: 223 laser scan samples

This discrepancy caused a state representation mismatch, making the trained policies incompatible with real sensor inputs.

G. Resolution and Model Adaptation

To address this:

- The simulation was updated to match the TurtleBot3 LiDAR configuration (223 samples)
- SAC and TD3 agents were retrained in simulation with the updated state representation
- The retrained policies were successfully deployed on the physical robot

H. Retrained Model Training Curves (Simulation)

Training curves for the retrained models (SAC and TD3) were recorded to demonstrate convergence and stability after sensor-aligned retraining. Metrics included:

- Success rate
- Cumulative reward
- Actor and critic loss

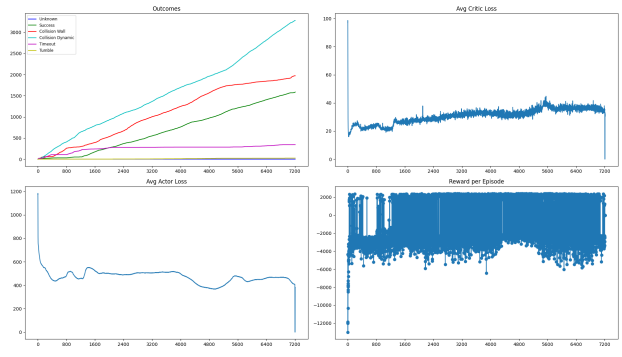

Fig. 13. Retrained SAC agent training curves: success rate, cumulative reward, actor loss, and critic loss after LiDAR alignment.

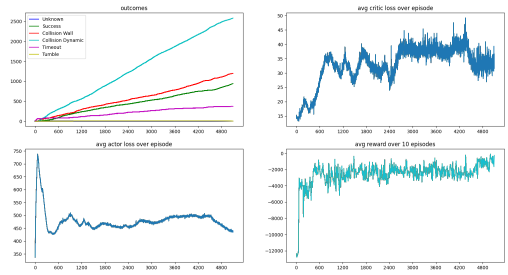

Fig. 14. Retrained TD3 agent training curves: success rate, cumulative reward, actor loss, and critic loss after LiDAR alignment.

The retrained models exhibit improved stability and convergence compared to the initial training configuration, highlighting the importance of sensor consistency for sim-to-real transfer.

I. Real-World Observations

Qualitative observations from real-world tests include:

- Successful detection of walls and tables, with effective collision avoidance
- Chairs were sometimes undetected due to thin legs, resulting in occasional close passes
- TD3 exhibited stable, deterministic motion with smooth trajectories
- SAC showed higher exploration, causing minor oscillations

Despite environmental complexity, the robot navigated autonomously without manual intervention after retraining.

J. Interpretation and Insights

- Sensor-model alignment is critical for sim-to-real transfer; the initial failure demonstrates the impact of even small observation mismatches.
- TD3 and SAC successfully operated in real-world scenarios, though performance remained qualitative rather than optimal.
- Observed challenges include minor oscillations, incomplete detection of thin obstacles, and sensitivity to cluttered layouts.
- Recommendations for improved performance:
  - Refine reward scaling to penalize unsafe proximity more effectively
  - Adjust exploration parameters and SAC entropy coefficients to reduce oscillations
  - Extend domain randomization and training episodes for robustness to irregular obstacles
  - Fine-tune using limited real-world experience to reduce sim-to-real gap

Overall, the current models provide a functional baseline for real-world indoor navigation. With further tuning, they have potential for smoother, safer, and more reliable operation.

IX. Conclusion

The key outcomes of this study are summarized as follows:

1) A comprehensive deep reinforcement learning (DRL)–based navigation framework was developed for the TurtleBot3 robot using ROS2 and the Gazebo simulation environment. Multiple DRL algorithms—DQN, DDPG, TD3, SAC, and CrossQ—were implemented under identical conditions to allow fair and systematic performance comparison. The modular architecture, standardized training pipeline, and unified evaluation metrics ensured reproducibility and consistency.
2) Experimental results showed that continuous-action actor–critic methods outperform discrete-action approaches in goal-reaching capability. TD3 achieved the highest success rate and lowest collision frequency in simulation, although with longer paths and higher execution times. DDPG balanced success rate and efficiency but showed unstable control. SAC exhibited strong exploration and adaptability but higher collision rates. CrossQ emphasized exploration at the cost of efficiency, while DQN produced safe and efficient trajectories due to its conservative discrete control.
3) Real-world deployment on a physical TurtleBot3 validated the feasibility of DRL-based navigation and highlighted sim-to-real challenges. Initial failures caused by LiDAR observation mismatch emphasized the importance of sensor-model alignment. After retraining with corrected LiDAR inputs, both TD3 and SAC successfully navigated in real indoor environments, demonstrating goal-directed behavior and collision avoidance.
4) Qualitative observations in real-world experiments revealed limitations, including sensitivity to thin obstacles, minor oscillations in motion, and reduced robustness in cluttered environments. These insights suggest the need for additional fine-tuning, reward scaling, and extended training to improve generalization.
5) Overall, this study confirms that DRL is a viable approach for autonomous robot navigation. The proposed framework serves both as a practical navigation system and a research platform. Future improvements, including extended training, refined reward shaping, enhanced domain randomization, and limited real-world fine-tuning, can further enhance navigation performance and robustness in complex real-world scenarios.

References

[1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, 2018, pp. 1861–1870.

[2] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Juan, Puerto Rico, 2016.

[3] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, 2018, pp. 1587–1596.

[4] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.

[5] R. Mitchell, J. Wang, and S. Levine, "CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and stability," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Honolulu, HI, USA, 2023.
[6] M. Quigley *et al.*, "ROS: An open-source robot operating system," in *ICRA Workshop Open Source Softw.*, Kobe, Japan, 2009, p. 5.
[7] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sendai, Japan, 2004, pp. 2149–2154.
[8] Open Robotics, "Ignition Gazebo: Next-generation robotics simulation," 2023. [Online]. Available: https://gazebosim.org
[9] ROBOTIS, "TurtleBot3: Open-source mobile robot platform," 2023. [Online]. Available: https://emanual.robotis.com/docs/en/platform/turtlebot3/
[10] Open Robotics, "ROS 2 Humble Hawksbill documentation," 2023. [Online]. Available: https://docs.ros.org/en/humble/
[11] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8026–8037.
[12] A. Raffin *et al.*, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
[13] G. E. Uhlenbeck and L. S. Ornstein, "On the theory of the Brownian motion," *Phys. Rev.*, vol. 36, pp. 823–841, 1930.
[14] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014.
[15] Canonical Ltd., "Ubuntu 22.04 LTS (Jammy Jellyfish)," 2022. [Online]. Available: https://ubuntu.com
[16] T. Vršek, *turtlebot3_drlnav: Deep reinforcement learning navigation for TurtleBot3*, GitHub repository, 2023. [Online]. Available: https://github.com/tomasvr/turtlebot3_drlnav
[17] J. Tobin *et al.*, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Vancouver, BC, Canada, 2017, pp. 23–30.